adaptec

Software RAID vs. Hardware RAID

Introduction

RAID implementations contain components such as RAID tables defining the configuration of RAID arrays, data structures to store the descriptors for cached data, engine(s) for calculating parity and the logic for handling I/Os to and from RAID arrays. These components may be implemented in software – typically in kernel-mode – or embedded in the controller for the secondary storage devices using which the RAID arrays are created. Which alternative is better? This paper answers that question by presenting an analysis of the issues associated with both alternatives and their performance in a real-world environment.

RAID in Software

Mainstream system processors continue to evolve on a very aggressive curve. We have come a long way since Intel's introduction of the first modern microprocessor in 1982. Its 80286 with 134,000 transistors achieved speeds of 12 MHz and delivered up to 41K FLOPS with the assistance of its 80287 co-processor. Intel's flagship (at the time this article was written), the Pentium 4 with 42,000,000 transistors, achieves a blazing 1.7 GHz, and delivers up to 900 MFLOPS. This growth – and the implicit assurance of enhanced performance by the use of succeeding generation of processors – has enticed developers to place greater loads on system CPUs with a menagerie of applications. Software based RAID is one of them.

However, there are some drawbacks to implementing RAID in software. First is the issue of portability. Since a software implementation will undoubtedly have OS-specific components, those components will have to be re-written for each OS. The second issue is the one that haunts kernel-mode software developers. Kernel-mode programs have to be perfect. Unlike applications, their ability to execute privileged instructions and manipulate the contents of any virtual address leaves the system without any safeguards against programming errors. The consequence can be a crashed system!

System CPU Load

As we have already stated, software RAID solutions are typically implemented as kernel-mode components. In fact under Linux it is incorporated into the kernel itself. How does that impact the CPU? Most kernel mode components avoid spawning threads to avoid the costly overhead of context switching. However, kernel mode components are still at the mercy of the scheduler that preempts their operation as soon as their time quantum expires or a higher priority task is scheduled. Thus, even under the most hospitable circumstances, a kernel-mode RAID engine is compelled to share processor time with other kernel mode components and the overlying applications that use them. This may not be critical if those applications are docile with respect to their processing needs. However, certain applications (and their underlying drivers) and environmental factors can overwhelm the CPU. Let us look at some of the them.

Network Traffic

Servers by their intrinsic nature are networked to provide services to clients over a network. For this reason the effect of network traffic on servers is of significance. Network interface cards (NIC) are heavily reliant on the system CPU for protocol-specific processing and transferring data to and from physical memory. In fact, they consume a disproportionately large amount of CPU time in view of this dependency. This section presents a picture of how NICs work and interact with drivers in a system.

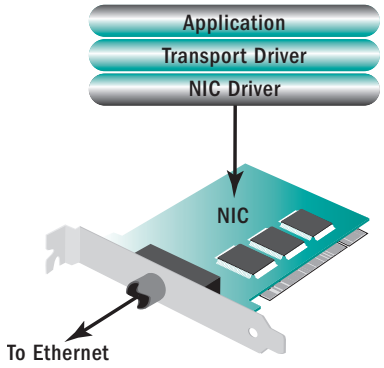


Figure 1 – Hierarchy of Network Drivers

NICs are managed by NIC drivers. Such drivers perform functions such as handling interrupts from the NIC, receiving and sending packets to and from the network, and also providing an interface to set or query operational characteristics of the NIC. An NIC driver typically interfaces with a transport driver above it. A transport driver implements the stacks for network protocols such as TCP/IP or IPX/SPX. It successively strips and interprets the network-protocol layers of the packets handed to it by the NIC driver and transfers the data contained in the "stripped" packets to system memory. Conversely, it wraps data supplied to it by the overlying application with suitable layers required by the network-protocol and hands it off to the NIC driver for transmission. *Figure 1* displays the network driver hierarchy. These drivers handle the bulk of the tasks involved in processing network packets, and since these drivers are executed in the system's CPU, that CPU bears the entire associated processing burden. How severe is this burden? To answer this question, consider a client server application built atop sockets using TCP, and the important processor intensive steps that the network drivers must take for such an application to function correctly.

- The use of TCP based sockets implies guaranteed delivery of each transmitted packet without loss of integrity. Packets can be easily lost or garbled during transmission. Therefore the network drivers at the receiving system must request re-transmission when necessary and the network drivers at the transmitting system must have the appropriate mechanism to comply with such requests.
- The individual packets must be sequenced. The transport driver at the receiving system must re-sequence the packets in the correct order to reconstruct the original data stream.
- The data content of each received packet must be copied to system memory at the receiving system. Note that, DMA is generally not an option available on NICs; hence operations to copy data to system memory requires the system CPU to be interrupted and used to execute the operation – a process commonly known as Programmed I/O (PIO). Conversely, data supplied by applications on the transmitting system has to be copied into network packets constructed appropriately for transmission. Furthermore since the size of data packets are restricted (though configurable) by each protocol to a size of approximately 1 KByte, it implies frequent CPU interruptions when the quantity of data being transmitted is large.

Clearly, these steps provide a good qualitative picture of the burden placed on the system CPU by network traffic. Now to get a quantitative picture of this scenario, we recommend a little experiment to the reader. Log into a NT or Windows 2000 system that has a network card and is attached to your intranet. Fire up Performance Monitor which is a standard administrative tool shipped with the OS. Within Performance Monitor, switch to the "Chart" view if Performance Monitor does not already display it. Add the counters ***% Interrupt Time*** and ***% DPC Time*** to this view. These represent the percentage of CPU time taken to service hardware interrupts and DPCs. Now select some files on any server on the network and copy them on to your local hard drive(s). It would be preferable if the amount of data is large – 100 MByte or more – so that you can get Performance Monitor to display the values for the aforementioned counters over a longer span of time. Note down the approximate median values for the counters. It should not come as a surprise if the approximate values for your ***% Interrupt Time*** is (or exceeds) 10%, and that for the ***% DPC Time*** is (or exceeds) 25%! In other words, about a third (or more) of your processors time is spent in being interrupted and completing I/Os. This experiment should convince the user of the expense involved in processing network traffic.

Application

While the applications driving file and print servers have a negligible impact on the CPU, application servers tend to impact the CPU severely. To understand why, let us take a look at the nature of application servers. Typically application servers are the back-end of complex business applications that satisfy the following requirements – high-availability, high-performance and redundancy.

Consider an application server that envelops a relational database. Anyone familiar with relational databases is acutely aware of the computational expense of performing many of the standard operations. Operations such as inner joins – in mathematical terms – have an order of $O(mn)$ where m and n are the size of the record sets. Furthermore these results cannot be preprocessed since the record sets for most applications are dynamic, i.e., they change with time. As a consequence their demand on computing resources is enormous.

OS Architecture and Components

The architecture of the OS can play a role in affecting CPU load. While a high degree of modularity ensures robustness and facilitates ease of maintenance of OS components, it also introduces performance latency at inter-module interfaces. Furthermore, the efficiency of implementations for open standards can vary from one OS to another. For instance, comparisons of CPU utilization using identical NICs and applications on Netware and NT often displays disparity in performance that can be attributed to one or both of the following factors – the relative efficiency of the NDIS implementations and the relative degree of modularity of the operating systems.

In summary, the load on the system CPU can be substantial due to the aforementioned factors even when discounting I/O processing to and from secondary storage. Clearly, there is a need to employ auxiliary processors to execute that role and relieve the system CPU of the additional burden. Let us now take a look at hardware RAID in detail and illustrate some of the salient aspects of its architecture that enhances performance.

RAID in Hardware

There are several advantages to implementing RAID in hardware. Let us first take a look at embedded processors that are at the heart of hardware RAID solutions. What is their horsepower? Though embedded processors are designed to be application-centric, any mainstream processor can be used for embedded development. In fact, the cores for embedded processors are usually related (if not identical) to their mainstream counterparts. Consequently, the upper bound of their processing power is no less than that for the mainstream ones. However, in practice, embedded processors are generally several orders of magnitude slower than mainstream processors. Why? It is usually a function of price. Embedded processors are designed to address the needs of specific applications, and are not expected to perform the generalized role of mainstream processors. It is this niche role that usually imposes restrictions on their price, and in turn on the horsepower that can be strapped on to them.

Is hardware RAID more efficient than software RAID? The answer is yes. First, the RAID firmware is executed on a dedicated processor and therefore does not share the system's CPU(s) with other kernel mode components and the overlying applications that use them. This has all the advantages of asymmetric multi-processing. Second, it is portable across operating systems and in the event of a malfunction in the RAID hardware or firmware, the server can usually continue to operate and even inform the user of the malfunction (assuming that there is a watchdog implementation in place). Conversely, if the server crashes due to some unexpected event, hardware RAID generally offers better survivability. Many hardware RAID solutions are armed with battery backup modules that allow them to maintain the coherency of their caches and complete outstanding operations without loss of integrity. Finally, one of the great advantages offered by hardware RAID is the fact that, the arena of embedded development is centered on the principle of specialization for a target application. Consequently, hardware RAID often incorporates features that are specialized for optimizing performance.

Examples of such specialized features include the following.

- Use of auxiliary processor(s) dedicated to calculating the parity for data blocks that are to be written to disk while the main embedded processor is concurrently fetching or executing the next instruction in the RAID (firmware) code. This hardware component is not found on non-RAID HBAs.
- Use of dedicated cache(s) on the controller for reading or writing data. While the advantage offered by the use of a cache for reading is rather obvious, the advantage when writing may warrant a little explanation. A cache offers the host the opportunity to transparently complete "write" commands even while the read-write heads on the disk to which the command is targeted is seeking the appropriate sector(s) for writing the associated data. This obviates the need to interrupt the host and notify it when a desired sector has been sought by the read-write head permitting it to execute a write operation. Additionally, it also allows the controller to coalesce contiguous "dirty" data blocks that have accumulated over time, and write them out in a consolidated chunk. Clearly, this has the advantage of reducing the time spent in seeking the appropriate sectors on disks into which to write the individual blocks.

Performance Results

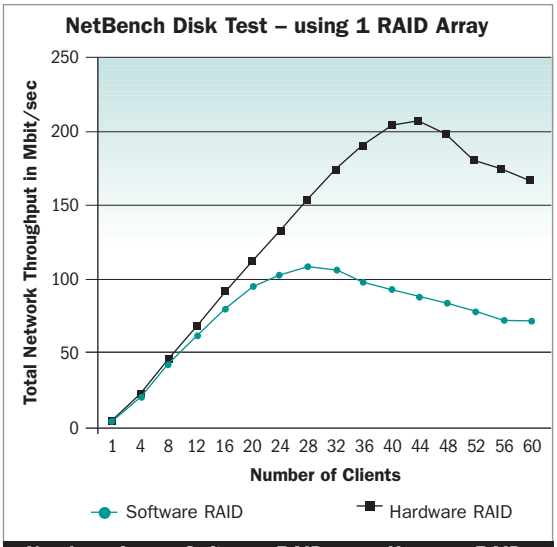
To obtain a quantitative picture of the superiority in performance of hardware RAID to software RAID consider the following performance test results obtained from using NetBench Disk test (version 7.0). NetBench is an application that measures the performance of file servers handling network file requests from clients running Windows® 95/98, Windows NT® or Windows 2000.

Two sets of NetBench Disk tests were conducted on RAID 5 arrays, the first set utilizing one array comprised of six disks and the latter utilizing two arrays comprised of six disks each. The Adaptec SCSI RAID 3210S – a mid-range SCSI controller – with 64 MByte of on-board RAM was pitted as a representative of hardware RAID against the native software RAID utility provided by Windows 2000 server used in conjunction with an Adaptec 39160 SCSI card. *Table 1* displays the configuration details and *Figure 2* the corresponding cumulative network throughput for the first test. *Table 2* displays the configuration details and *Figure 3* the corresponding cumulative network throughput for the second test.

Note that these tests are intended to illustrate the general superiority of hardware RAID to software RAID and the use of a mid-range controller for hardware RAID is sufficient for that purpose. Certainly the use a high-end hardware RAID controller can be expected to amplify this superiority further.

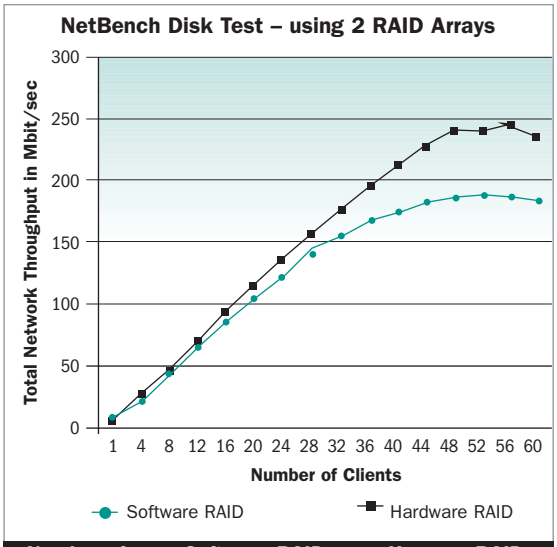
Operating System	Windows 2000 Server	
System Memory	1 GByte, PC133	
RAID Type	RAID 5	
Number of Drives	6	
Drive Type	Seagate ST318451LC, 15K rpm, 18.35 GByte	
Number of Arrays	1	
NIC	Intel PRO/1000 T Server Adapter, 1 GBit	
	Hardware RAID	**Software RAID**
Controller	Adaptec 3210S	Adaptec SCSI Card 39160
SCSI Interface	Ultra160	Ultra160
Available Channels	2	2
Channels Used	1	1

Table 1 – Test Configuration



Number of Clients	Software RAID Mbit/sec	Harware RAID Mbit/sec
1	5.6	5.8
4	22.3	23.1
8	43.3	40.0
12	63.2	69.0
16	81.0	91.2
20	96.1	113.0
24	103.8	134.3
28	109.5	154.3
32	107.4	175.7
36	98.6	190.3
40	94.6	204.5
44	90.2	208.0
48	85.7	198.1
52	80.1	180.8
56	74.0	174.4
60	73.8	167.1

Figure 2 – Software vs. Hardware RAID Performance Using 1 RAID Array



Number of Clients	Software RAID Mbit/sec	Harware RAID Mbit/sec
1	5.4	5.7
4	21.5	23.1
8	41.1	46.1
12	65.4	68.8
16	86.2	91.6
20	105.9	113.7
24	123.9	134.7
28	140.8	156.4
32	156.9	175.2
36	169.5	195.8
40	175.9	211.4
44	183.6	228.0
48	188.4	239.7
52	190.0	240.9
56	188.0	245.6
60	185.4	236.3

Figure 3 – Software vs. Hardware RAID Performance Using 2 RAID Array

Operating System	Windows 2000 Server	
System Memory	1 GByte, PC133	
RAID Type	RAID 5	
Number of Drives	6 per Array	
Drive Type	Seagate ST318451LC, 15K rpm, 18.35 GByte	
Number of Arrays	2	
NIC	Intel PRO/1000 T Server Adapter, 1 Gbit	
	Hardware RAID	**Software RAID**
Controller	Adaptec 3210S	Adaptec SCSI Card 39160
SCSI Interface	Ultra160	Ultra160
Available Channels	2	2
Channels Used	2	2

Table 2 – Test Configuration

Conclusion

Hardware RAID is a superior solution to software RAID in a networked environment as is typical for servers. Its benefits are even more significant when running applications with high CPU utilization.

Glossary

Application Server
An application server is the engine that acts as the intermediary for data and services between a "thin" web-enabled client in the front-end and a database or repository of some form in the back-end. This may include web-servers, OLTP servers etc.

Asymmetric Multi-Processing
Multi-processing using two or more processors that are not equivalent in their capabilities and their use.

Cache
A part or whole of a dynamic memory space that is used to store data being written to secondary storage and subsequently read from it.

Context Switch
The action by which the state information for a process whose execution is stopped (by the scheduler) are swapped out and that for a dormant process that is to begin execution are swapped in.

CPU
Central Processing Unit (of which a system may have one or more).

Dirty Data
Data that is residing in cache but has not been written to its target (such as secondary storage).

DMA
Direct Memory Access. Methodology by which an auxiliary processor transfers data between a peripheral device and the system memory without the intervention of the system's main CPU(s).

DPC
Deferred Procedure Call. It is a software routine that is part of a driver invoked when an I/O is completed. I/O completion typically involves checking I/O status, forwarding I/Os (returned by the underlying drivers) to overlying drivers in a layered driver model and executing cleanup actions that may be necessary.

Embedded
In conjunction with the terms processor or development refers to the area of specialized applications that typically run on a single micro-processor board with the program residing in flash memory.

Inner Join
Combines records from two tables whenever there are matching values in a common field

Kernel
The central component of an operating system that is typically responsible for memory, process, security and I/O management.

Multi-Processing
Multi-processing is the division of labor in computing, with each processor executing a distinct set of tasks. If the set of tasks being executed by one processor is reasonably independent of the set of tasks being executed by another (or vice-versa) then multi-processing can yield significant performance gains.

NDIS
Network Driver Interface Specification. It is the specification for the interface between device drivers and a network. All transport drivers call the NDIS interface to access and work with NICs.

O(n)
Pronounced as "order of n". If an algorithm (or heuristic) dependent on the variable n has a complexity of ***O(n)***, then the algorithm (or heuristic) takes time proportional to ***n*** to complete execution.

Outer Join
Simple union of all records from two tables.

Physical Memory
Dynamic memory or simply random access memory (RAM).

PIO
Methodology by which I/O transfers to and from secondary storage are performed by the system CPU.

RAID
Redundant Array of Inexpensive Disks. Methodology using which multiple disks are coalesced to form an array that provides redundancy and higher availability of data.

Relational Database
Database that employs multiple "related" tables for storing data.

Scheduler
Component of the OS kernel that controls the order and time of execution of processes and their associated threads.

Virtual Address
Address that is not necessarily backed up by physical memory. Typically the virtual address space is significantly larger than the physical memory size, and is backed up by on-disk space.

Watchdog
An application which "watches" over specified target component(s). Typically a watchdog performs a set of diagnostic checks at pre-specified intervals on its target component(s), and perform suitable action depending on the status of its target.

adaptec®

Copyright 2002 Adaptec, Inc. All rights reserved. Adaptec and the Adaptec logo are trademarks of Adaptec, Inc. which may be registered in some jurisdictions. Microsoft, Windows, Windows NT, Windows 95/98/2000 are trademarks of Microsoft Corporation, used under license. All other trademarks used are owned by their respective owners.

P/N 666261-011 Printed in USA 2/02